PYTHON

Agentes RAG com Python: como criar um assistente de IA

> Aprenda a criar Agentes RAG com Python do zero: LangChain, FAISS e OpenAI para montar um assistente de IA que responde com base nos seus documentos.



Diego Monutti · Analista de Operações e Especialista em Python

Publicado em 17 de setembro de 2026

hashtagtreinamentos.com

Neste material

Agentes RAG com Python: como criar um assistente de IA

1. Introdução
2. O que é um sistema RAG e por que ele vale tanto no mercado
   - Como funciona o RAG na prática
   - Por que não simplesmente usar o ChatGPT ou Gemini direto
3. Preparando o ambiente para criar o agente RAG
   - Arquivos necessários para o projeto
   - Instalando as bibliotecas com requirements.txt
   - Configurando a chave da API da OpenAI
4. Entendendo a interface criada com Streamlit
5. Criando a lógica do agente de IA com LangChain
   - Montando a chain com prompt, contexto e pergunta
   - Testando o agente com um contexto simples
6. Construindo o sistema RAG completo
   - Carregando o PDF com PyPDFLoader
   - Dividindo o documento em blocos de texto
   - Criando o banco de vetores com embeddings e FAISS
   - Conectando o retriever ao agente
7. Testando o agente RAG completo na prática
8. Como aplicar esse agente RAG na sua empresa ou projeto
9. Conclusão

10. FAQ – Perguntas Frequentes sobre Agentes RAG com Python

1. O que é um agente RAG?

2. Preciso saber muito Python para criar um agente RAG?

3. É possível usar outro modelo de IA em vez da OpenAI?

Aprenda a criar Agentes RAG com Python do zero: LangChain, FAISS e OpenAI para montar um assistente de IA que responde com base nos seus documentos.

Introdução

Se você já usou o ChatGPT, o Gemini ou o Claude para responder perguntas, sabe o quanto isso é útil no dia a dia. Mas criar **agentes RAG com Python** é outro nível de aplicação, porque em vez de você usar a IA como ferramenta pessoal, você constrói um produto que usa a IA como parte da solução. É exatamente isso que você vai aprender aqui, na prática, com código completo e explicado do início ao fim.

Vamos construir juntos um assistente de RH que responde perguntas sobre políticas internas de uma empresa, tipo “posso trabalhar home office todos os dias?” ou “como funciona a licença paternidade?”. A resposta não vem de qualquer lugar da internet, ela vem exclusivamente de um documento PDF que você vai fornecer para o sistema. Isso é um sistema RAG funcionando de verdade, com Python, LangChain, FAISS e a API da OpenAI.

Ao final deste conteúdo, você vai entender como montar essa estrutura do zero, entender por que ela é mais barata e mais segura do que simplesmente jogar um PDF inteiro para a IA, e vai ter um projeto funcional rodando no seu computador, pronto para ser adaptado para qualquer outro tipo de documento ou negócio.

O que é um sistema RAG e por que ele vale tanto no mercado

RAG vem de retrieval augmented generation, ou geração aumentada por recuperação. O nome é feio, mas a ideia é simples. É um sistema que responde perguntas com base em uma informação que é sua, um PDF, uma base de dados, uma planilha, qualquer fonte de conteúdo que você queira usar como referência para a IA responder.

No nosso exemplo, quem alimenta o sistema é um PDF com as políticas internas de RH de uma empresa fictícia. Mas o mesmo raciocínio serve para manuais de produto, contratos,

catálogos, base de conhecimento de atendimento ao cliente, entre outros. Se você já pesquisou sobre o que é RAG, vai perceber que essa técnica é hoje uma das mais usadas para colocar inteligência artificial dentro de produtos reais.

Como funciona o RAG na prática

Pensa assim: quando você manda uma pergunta para o sistema, duas coisas acontecem ao mesmo tempo. A pergunta vai para a inteligência artificial, mas ela também vai para a sua base de dados, para buscar apenas os pedaços de texto que têm relação com aquilo que você perguntou. Depois, só esses pedaços relevantes são enviados junto com a pergunta para a IA responder.

Etapa	O que acontece
1. Pergunta do usuário	O texto digitado no chat é capturado pelo sistema
2. Busca no banco de vetores	A pergunta é comparada com os blocos de texto do documento
3. Montagem do contexto	Os blocos mais relevantes são selecionados como contexto
4. Resposta da IA	O modelo responde com base apenas no contexto encontrado



Por que não simplesmente usar o ChatGPT ou Gemini direto

Você poderia simplesmente subir o PDF direto no ChatGPT e perguntar por ali. Só que isso cria dois problemas bem concretos para quem quer transformar isso em um produto de verdade.

- Aquela conversa fica presa dentro do ChatGPT ou do Gemini, você não consegue integrar isso com o seu site, seu WhatsApp ou seu aplicativo, e nem permitir que várias pessoas conversem ao mesmo tempo com a mesma base de conhecimento.
- Ao mandar o documento inteiro toda vez que alguém pergunta algo, você paga por muito mais tokens do que precisaria, porque a IA lê o PDF inteiro mesmo quando a resposta está em apenas três linhas dele.

O sistema RAG resolve os dois problemas de uma vez: ele te dá liberdade para colocar a solução onde você quiser e reduz o custo, porque só envia para a IA os pedaços do documento que realmente importam para aquela pergunta específica.

Preparando o ambiente para criar o agente RAG

Antes de sair escrevendo código, você precisa organizar os arquivos do projeto e instalar as bibliotecas certas. Se você ainda não tem familiaridade com a sintaxe básica do Python, vale a pena dar uma passada por um curso básico de Python antes de seguir, porque esse projeto assume que você já entende como funcionam funções, variáveis e importações.

Arquivos necessários para o projeto

São três arquivos que você precisa ter dentro de uma mesma pasta no seu editor de código, seja o VS Code, o PyCharm ou o que você preferir usar.

Arquivo	Função
app_basico.py	Código inicial com a interface visual do chat feita em Streamlit, sem lógica de IA
politica_rh.pdf	Documento que vai alimentar o sistema RAG com as informações da empresa
requirements.txt	Lista de bibliotecas que precisam ser instaladas para o projeto funcionar

Instalando as bibliotecas com requirements.txt

O arquivo requirements.txt reúne todas as bibliotecas que o projeto precisa. É assim que o conteúdo dele fica:

```
streamlit
langchain
langchain-community
langchain-openai
langchain-text-splitters
faiss-cpu
pypdf
python-dotenv
```

Com os arquivos dentro da pasta, abra o terminal do seu editor e rode o comando abaixo para instalar tudo de uma vez, sem precisar decorar nada disso:

```
pip install -r requirements.txt
```

Se você tiver conexão com a internet, esse comando vai ler o arquivo e instalar cada uma das bibliotecas listadas. Vale reforçar esse hábito, porque é assim que praticamente todo projeto Python profissional organiza suas dependências, e você pode aprofundar mais sobre isso lendo sobre como usar o pip install no Python.

Configurando a chave da API da OpenAI

Para o modelo de IA e o processo de embeddings funcionarem, você precisa de uma chave de API da OpenAI. O passo a passo é direto: entre na plataforma de API da OpenAI, faça login, vá em configurações, clique em API keys e crie uma nova chave, dando um nome qualquer para ela, como “agente rh”. Copie o código gerado.

Agora, dentro da pasta do projeto, crie um arquivo chamado `.env`. Esse tipo de arquivo guarda variáveis de ambiente, ou seja, informações sensíveis que você não quer deixar espalhadas dentro do código, como senhas e chaves de API. Dentro dele, cole a chave assim:

```
OPENAI_API_KEY=sua_chave_aqui
```

Esse nome precisa ser exatamente esse, porque é o padrão que a própria OpenAI define para essa variável. Depois, dentro do código Python, basta carregar essa variável usando a função `load_dotenv()`, algo que fica mais claro quando você entende bem como funcionam as variáveis de ambiente no Python.

Entendendo a interface criada com Streamlit

O arquivo inicial do projeto já vem com a interface visual pronta, feita com Streamlit, uma biblioteca do Python que permite criar chats, dashboards e telas de forma simples, com poucas linhas de código. Ela não tem nenhuma lógica de IA ainda, só a estrutura visual da

conversa.

Para rodar esse tipo de aplicação, você não clica em “executar” o arquivo Python como faria normalmente. Você abre o terminal e roda o comando abaixo, trocando o nome do arquivo pelo seu:

```
streamlit run app_basico.py
```

Isso abre um link local no navegador com o chat funcionando. A lógica de mensagens é simples: cada mensagem é armazenada como um dicionário com dois campos, um chamado `role`, que indica quem mandou a mensagem (usuário ou assistente), e outro chamado `content`, que é o texto em si. Essas mensagens ficam guardadas na sessão do usuário usando o `st.session_state`, o que garante que a conversa continue aparecendo enquanto a página estiver aberta.



Criando a lógica do agente de IA com LangChain

Agora começa a parte que realmente importa: dar inteligência para esse chat. Para isso, vamos usar o LangChain, que é a principal ferramenta em Python para criar esse tipo de cadeia de processamento entre o prompt do usuário e a resposta final da IA. Antes de mexer no sistema RAG completo, vale entender a lógica com um exemplo simplificado.

Primeiro, importe todas as ferramentas que vamos usar ao longo do projeto:

```
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

load_dotenv()
```

Montando a chain com prompt, contexto e pergunta

No LangChain, uma chain nada mais é do que uma sequência de passos conectados pelo símbolo de barra vertical. Cada passo recebe o resultado do passo anterior e passa adiante. No nosso caso, o fluxo é: montar o contexto e a pergunta, encaixar isso em um template de prompt, mandar para o modelo de linguagem e, por fim, extrair só o texto da resposta.

```
prompt_template = ChatPromptTemplate.from_template(
    """Você é um assistente de RH que responde perguntas sobre políticas internas da empresa.
Use APENAS as informações do contexto abaixo para responder.
Se não encontrar a resposta, diga claramente que não sabe responder.
Responda em português do Brasil, de forma clara e objetiva.

Contexto: {context}

A pergunta: {question}

Resposta:""")

llm = ChatOpenAI(model="gpt-4o-mini")

chain = (
    {"context": criar_contexto, "question": RunnablePassthrough()}
    | prompt_template
    | llm
    | StrOutputParser()
)
```

Repare que a variável `context` e a variável `question` dentro do template precisam ter exatamente o mesmo nome usado no dicionário da chain. O `RunnablePassthrough()` é só um jeito de dizer “pegue o que o usuário mandou e repita aqui sem alterar nada”, ele preenche a pergunta automaticamente. Já o `StrOutputParser()` serve para pegar a resposta bruta da IA, que normalmente vem como um objeto cheio de metadados, e transformar em um texto limpo.

Testando o agente com um contexto simples

Para validar a lógica antes de complicar com PDF e busca vetorial, dá para testar com uma função de contexto fixa, tipo um texto curto escrito à mão. O importante aqui é que toda função usada dentro da chain precisa obrigatoriamente receber o prompt do usuário como parâmetro, mesmo que ela não use esse valor para nada.

```
def criar_contexto(prompt_usuario):
    return "A política da empresa é home office para 100% das pessoas e férias precisam de aprovação prévia

resposta = chain.invoke("Posso tirar férias com 15 dias de aviso prévio?")
print(resposta)
```

Com esse teste rodando certinho, você já tem um agente que responde com base em um contexto. O problema é que, do jeito que está, esse contexto precisaria conter o documento inteiro para funcionar de verdade, o que nos leva direto para o próximo problema a resolver: como transformar um PDF de várias páginas nesse contexto de forma inteligente e barata.

Construindo o sistema RAG completo

Chegou a parte que transforma esse chat simples em um verdadeiro sistema RAG em Python. Isso acontece em três etapas: carregar o texto do documento, dividir esse texto em blocos e transformar esses blocos em vetores de números para permitir busca por significado.

Carregando o PDF com PyPDFLoader

O primeiro passo é extrair todo o texto do PDF. Para isso, usamos o `PyPDFLoader`, do próprio LangChain, que consegue ler arquivos PDF com Python sem complicação:

```
def carregar_pdf():
    pdf_loader = PyPDFLoader("politica_rh.pdf")
    texto = pdf_loader.load()
    return texto

texto_pdf = carregar_pdf()
```

Se você der um print nessa variável, vai ver todo o conteúdo do documento carregado, com título, seções e parágrafos. É esse texto completo que ainda vamos precisar quebrar em pedaços menores.

Dividindo o documento em blocos de texto

Mandar o documento inteiro para a IA responder qualquer pergunta é caro e ineficiente. A solução é separar esse texto em blocos menores, para que só o pedaço relevante seja enviado como contexto. Para isso usamos o `RecursiveCharacterTextSplitter`, definindo um tamanho de bloco e uma sobreposição entre eles:

```
def separar_em_blocos(texto_pdf):
    separador = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200
    )
    lista_blocos = separador.split_documents(texto_pdf)
    return lista_blocos

lista_blocos = separar_em_blocos(texto_pdf)
```

O `chunk_size` define o tamanho de cada bloco em caracteres, e o `chunk_overlap` cria uma sobreposição entre um bloco e o próximo. Isso evita que uma seção do documento fique cortada ao meio, terminando em um bloco e começando em outro. Um valor comum e que funciona bem na maioria dos casos é 1000 de tamanho com 200 de sobreposição, mas vale

testar outros valores dependendo da estrutura do seu documento.



Criando o banco de vetores com embeddings e FAISS

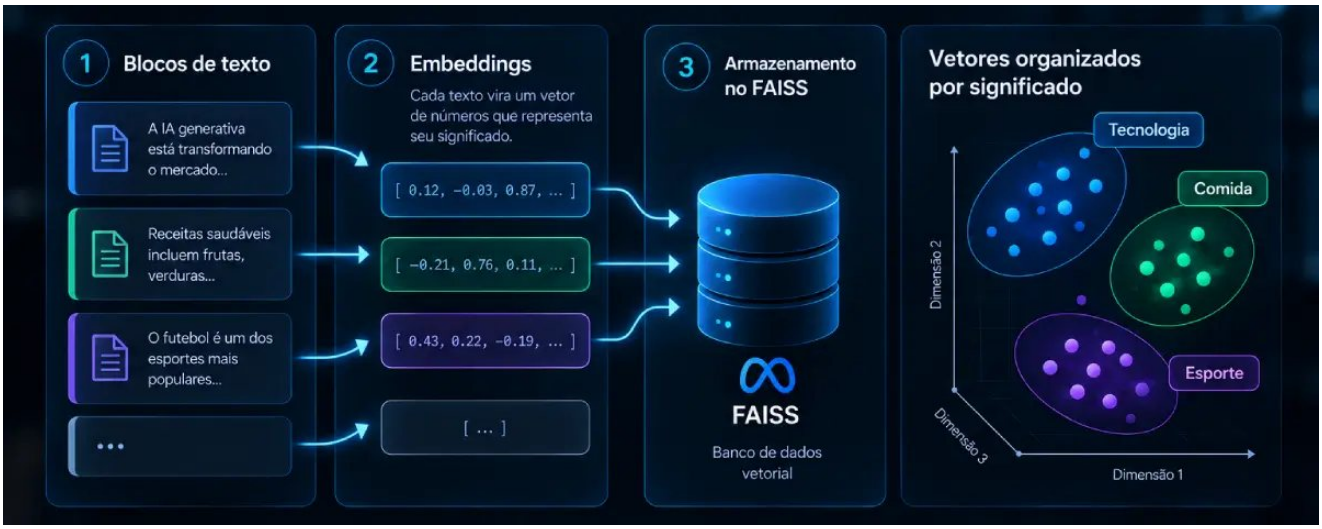
Aqui está o coração do sistema RAG. Cada bloco de texto é transformado em um vetor de números através de um processo chamado embedding. Textos com significados parecidos geram vetores próximos entre si, enquanto textos sobre assuntos diferentes ficam distantes. Isso permite comparar a pergunta do usuário com todos os blocos usando apenas cálculo matemático, o que é extremamente mais rápido e barato do que comparar texto com texto.

Para armazenar esses vetores usamos o FAISS, um banco de dados vetorial leve e gratuito, criado pela equipe de pesquisa de IA da Meta:

```
def criar_banco_vetores(lista_blocos):
    ferramenta_embedding = OpenAIEmbeddings(model="text-embedding-3-small")
    banco_vetores = FAISS.from_documents(lista_blocos, ferramenta_embedding)
    return banco_vetores

banco_vetores = criar_banco_vetores(lista_blocos)
```

Esse processo de transformar texto em vetor só precisa acontecer uma vez. Depois de criado, o banco de vetores fica pronto para receber qualquer pergunta e devolver os blocos mais relevantes em questão de milissegundos.



Conectando o retriever ao agente

Agora falta ligar o banco de vetores à chain do agente. Em vez de criar uma função manual de busca de contexto, o próprio FAISS já oferece um método chamado `as_retriever()`, que faz essa busca automaticamente, comparando a pergunta do usuário com os vetores armazenados e retornando os blocos mais parecidos:

```
def criar_chain_agente(banco_vetores):
    prompt_template = ChatPromptTemplate.from_template(
        """Você é um assistente de RH que responde perguntas sobre políticas internas da empresa.
Use APENAS as informações do contexto abaixo para responder.
Se não encontrar a resposta, diga claramente que não sabe responder.
Responda em português do Brasil, de forma clara e objetiva.

Contexto: {context}

A pergunta: {question}

Resposta:""")

    buscador_contexto = banco_vetores.as_retriever()
    llm = ChatOpenAI(model="gpt-4o-mini")

    chain = (
        {"context": buscador_contexto, "question": RunnablePassthrough()}
        | prompt_template
        | llm
        | StrOutputParser()
    )
    return chain

chain = criar_chain_agente(banco_vetores)
```

Por padrão, o `as_retriever()` devolve os quatro blocos mais parecidos com a pergunta. Esse número costuma ficar entre três e cinco na maioria dos projetos, porque garante contexto suficiente sem gastar tokens à toa. Se o agente estiver errando muitas respostas, vale aumentar essa quantidade. Se estiver acertando bem e você quiser economizar, dá para reduzir.

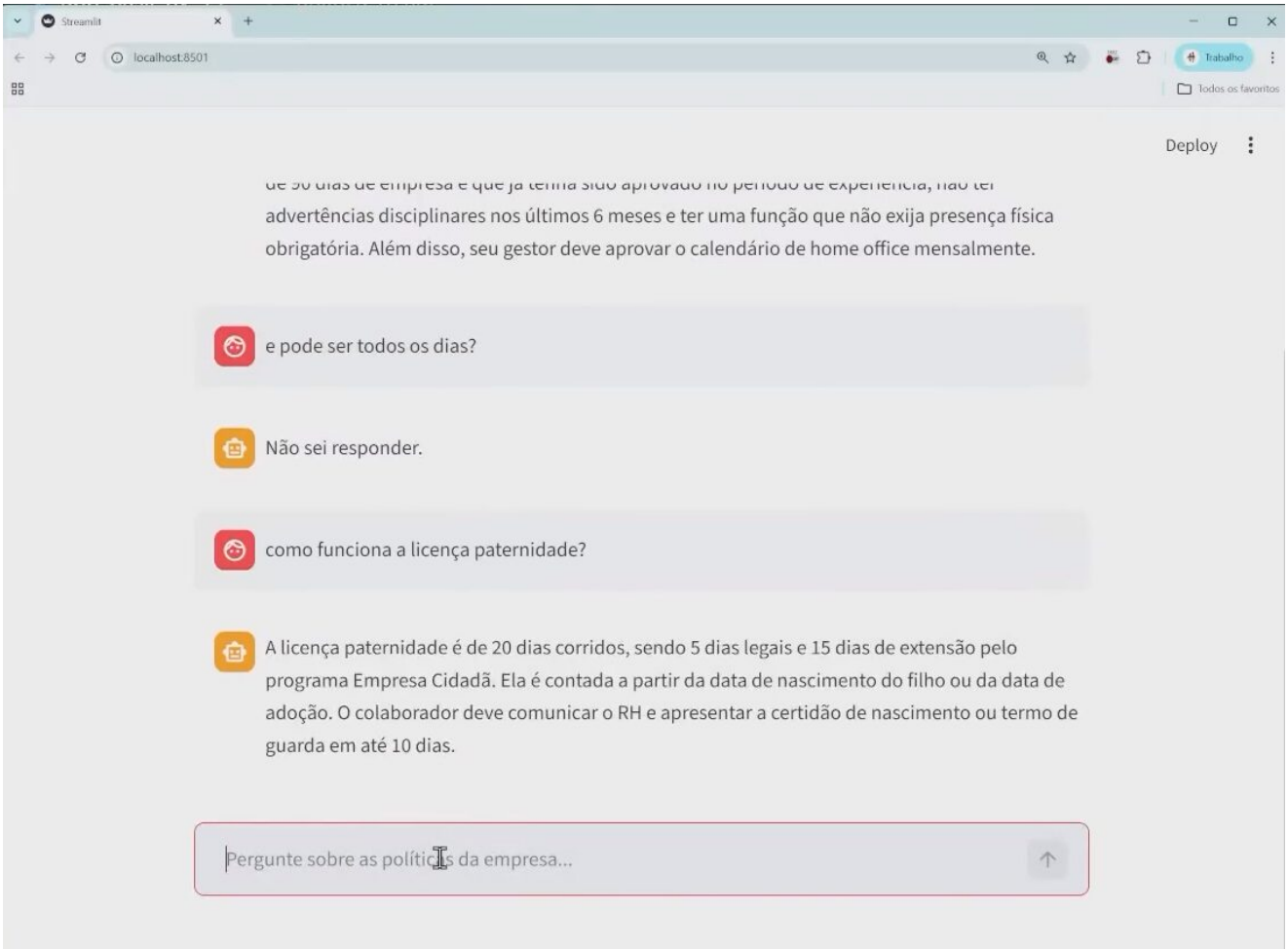
Testando o agente RAG completo na prática

Com tudo conectado, é hora de rodar o projeto de verdade. No terminal, execute novamente o comando do Streamlit apontando para o arquivo final:

```
streamlit run app_basico.py
```

Ao perguntar algo como “como funciona a licença paternidade?”, o sistema busca no PDF, encontra o bloco correto e responde com precisão, algo como 20 dias corridos, sendo 5 dias legais e 15 dias de extensão pelo programa Empresa Cidadã, contados a partir da data de nascimento ou adoção. Repare que essa resposta não veio da memória do modelo, ela veio exclusivamente do seu documento.

Se alguma pergunta não for respondida corretamente, o ajuste geralmente está em três lugares: a quantidade de blocos retornados pelo retriever, o tamanho dos blocos definidos no splitter, ou a forma como o documento está estruturado. Vale ir testando esses parâmetros até encontrar o equilíbrio ideal para o seu caso.



Como aplicar esse agente RAG na sua empresa ou projeto

A estrutura que você acabou de montar serve para muito mais do que um assistente de RH. Times de atendimento usam esse mesmo esquema para responder dúvidas de clientes com

base em manuais de produto. Times comerciais usam para tirar dúvidas sobre contratos e políticas comerciais. E você pode adaptar essa mesma lógica para qualquer agente de IA com Python que precise responder com base em uma fonte de informação específica.

Vale lembrar de alguns cuidados na hora de colocar isso em produção: pense na velocidade de resposta, em evitar alucinações e em se proteger de tentativas de prompt injection, quando alguém tenta manipular o comportamento do agente através da própria pergunta. Se você quiser ir além do Streamlit, o LangChain se integra tranquilamente com frameworks como FastAPI ou Flask, permitindo expor essa lógica como uma API para qualquer sistema, aplicativo ou site consumir.

Se você curtiu esse tipo de projeto e quer se aprofundar mais em como criar agentes de IA completos, vale também conhecer outras formas de integrar modelos de linguagem em Python, como construir um chatbot com ChatGPT em Python para outros tipos de uso.

Conclusão

Criar agentes RAG com Python deixa de ser um bicho de sete cabeças quando você entende a lógica por trás de cada etapa: carregar o documento, dividir em blocos, transformar em vetores e conectar tudo isso a um modelo de linguagem através de uma chain. O resultado é um sistema barato, rápido e que responde exatamente com base nas informações que você forneceu, sem depender do que a IA “acha” que sabe.

Esse é só o começo do que dá para construir usando Python junto com inteligência artificial. Se você quer aprender a criar projetos assim do zero, com uma trilha completa de programação, dados e IA aplicada, conheça o Python Impressionador e continue evoluindo na sua jornada com Python.

Caso prefira esse conteúdo no formato de videoaula, assista ao vídeo abaixo ou acesse o nosso canal do YouTube!



Vídeo: Vídeo do YouTube

youtu.be/XqG3RN6VzDw

FAQ – Perguntas Frequentes sobre Agentes RAG com Python

1. O que é um agente RAG?

É um sistema que responde perguntas com base em informações próprias, como um PDF ou uma base de dados, em vez de depender apenas do conhecimento geral da IA. Ele busca os trechos relevantes do documento e usa isso como contexto para gerar a resposta.

2. Preciso saber muito Python para criar um agente RAG?

Você precisa entender o básico da linguagem, como funções, variáveis e importações. A lógica do LangChain tem nomes um pouco confusos, mas as linhas de código em si são poucas e diretas.

3. É possível usar outro modelo de IA em vez da OpenAI?

Sim. O LangChain permite trocar facilmente o modelo usado, seja Claude, Gemini ou outro provedor. A estrutura da chain continua praticamente a mesma, mudando apenas a integração do modelo de linguagem.